

Fondamenti di Informatica

Gabriele Perna

Contents

1	Insiemi	5
1.1	Definizione	5
1.2	Notazioni	5
1.3	Descrizione e rappresentazione degli insiemi	5
1.4	Diagrammi di Eulero-Venn	5
1.5	Uguaglianza e inclusione tra insiemi	5
1.6	Disgiunzione	5
1.7	Insieme universo	5
1.8	Proprietà di uguaglianza e inclusione	6
1.9	Operazioni tra insiemi	6
1.10	Implicazioni logiche	6
1.11	Tipi di dimostrazione	6
1.12	Leggi degli insiemi	6
1.13	Cardinalità	6
1.14	Insiemi di insiemi	6
1.15	Insieme delle parti	7
1.16	Famiglie e partizioni di insiemi	7
1.17	Prodotto cartesiano	7
2	Relazioni tra insiemi	8
2.1	Definizione	8
2.2	Relazione identità	8
2.3	Composizione di relazioni	8
2.4	Relazione opposta	8
2.5	Proprietà delle relazioni (TUSI)	8
2.5.1	Relazioni totali	8
2.5.2	Relazioni univalenti	9
2.5.3	Relazioni suriettive (surgettive)	9
2.5.4	Relazioni iniettive	9
2.6	Risultati di dualità	9
2.7	Teorema di caratterizzazione	9
2.8	Composizione e proprietà	9
3	Funzioni	10
3.1	Definizione	10
3.2	Composizione di funzioni	10
3.3	Funzione parziale	10
3.4	Biiezione (Bigezione)	10
3.5	Teorema di categorizzazione delle biiezioni	10
3.6	Proprietà delle biiezioni	11
4	Induzione	12
4.1	Definizione induttiva di un insieme	12
4.2	Definizione induttiva di una funzione	12
4.3	Definizione induttiva di una proprietà	13

5	Relazioni su un insieme	14
5.1	Proprietà delle relazioni su un insieme (RATSO)	14
5.2	Teorema di caratterizzazione	14
5.3	Chiusure di una relazione	14
5.4	Relazioni particolari	15
6	Grafi orientati	16
6.1	Definizione	16
6.2	Rappresentazione di grafi orientati	16
6.3	Varianti di grafi	17
6.4	Connettività	17
6.5	Ordinamento topologico su DAG	18
7	Grafi non orientati	19
7.1	Definizione	19
7.2	Grafo orientato associato a un grafo non orientato	19
7.3	Rappresentazione di grafi non orientati	20
8	Alberi	21
8.1	Caratteristiche	21
8.2	Proprietà	21
8.3	Alberi importanti	22
9	Cammini e cicli Euleriani e Hamiltoniani	24
9.1	Teorema di Eulero	24
9.2	Teorema di Hamilton	24
9.3	Il problema dei sette ponti di Königsberg	25
9.4	Problema del commesso viaggiatore	25
10	Altro sui grafi	26
10.1	Distanze	26
11	Calcolo combinatorio	27
11.1	Cardinalità di operazioni su insiemi	27
11.2	Cardinalità di relazioni	27
11.3	Cardinalità dell'insieme delle parti	28
11.4	Le targhe delle macchine italiane	28
11.5	Permutazioni, disposizioni e combinazioni	28
11.5.1	Permutazione	28
11.5.2	Disposizioni	29
11.5.3	Combinazioni	29
11.5.4	Cardinalità sfruttando regole di biiezione	30
11.6	Calcolo combinatorio sui grafi	30
12	Induzione strutturale	32
12.1	Liste	32
12.2	Induzione strutturale su alberi binari	33
12.3	F-Terms	33
12.4	Principio di induzione strutturale	34

13 Ricorsione	35
14 Linguaggi Formali	36
14.1 Automa	36
14.2 Automi deterministici e non deterministici	37
14.3 Grammatiche	37
15 Logica	39
15.1 Tecniche di dimostrazione	40
15.2 Logica dei predicati	40

1 Insiemi

1.1 Definizione

Un insieme è una collezione di oggetti detti elementi. In un insieme non esiste un ordine di elementi e ogni elemento è presente oppure no. Non esiste molteplicità degli elementi.

1.2 Notazioni

- Lettere maiuscole: indicano insiemi (A, B)
- Lettere minuscole: indicano elementi (a, b)

Un insieme vuoto si indica con $\emptyset$.

Appartenenza:

$$a \in A \quad \text{“l'elemento } a \text{ appartiene all'insieme } A”$$

Definizione tramite parentesi graffe:

$$A = \{1, 2, 3, 4, 5, 11\}$$

1.3 Descrizione e rappresentazione degli insiemi

- Per enumerazione: si elencano gli elementi con le parentesi graffe.
- Per proprietà: si definiscono le proprietà che gli elementi devono soddisfare.

Template:

$$A = \{x \in B \mid P(x)\}$$

“L'insieme A contiene tutti gli elementi di B che soddisfano la proprietà P ”.

1.4 Diagrammi di Eulero-Venn

Utilizzati per rappresentare graficamente gli insiemi.

1.5 Uguaglianza e inclusione tra insiemi

- Due insiemi sono uguali se ogni elemento del primo appartiene al secondo e viceversa.
- Sottoinsieme: $A \subseteq B$ indica che tutti gli elementi di A appartengono anche a B .
 $A \subset B$ indica che A è sottoinsieme proprio di B .

1.6 Disgiunzione

Due insiemi sono disgiunti se non hanno elementi in comune.

1.7 Insieme universo

L'insieme universo U include tutti gli elementi nel contesto considerato.

1.8 Proprietà di uguaglianza e inclusione

- Riflessività: $A = A$, $A \subseteq A$, $A \subset A$ NON vale
- Transitività: se $A = B$ e $B = C$ allora $A = C$; se $A \subseteq B$ e $B \subseteq C$ allora $A \subseteq C$
- Simmetria: $A = B \implies B = A$ (valida solo per uguaglianza)
- Antisimmetria: $A \subseteq B$ e $B \subseteq A \implies A = B$

1.9 Operazioni tra insiemi

- Unione: $A \cup B = \{x \mid x \in A \vee x \in B\}$
- Intersezione: $A \cap B = \{x \mid x \in A \wedge x \in B\}$
- Differenza: $A \setminus B = \{x \mid x \in A \wedge x \notin B\}$
- Complemento: $\overline{B} = U \setminus B$ o $\neg B$

1.10 Implicazioni logiche

$P \Rightarrow Q$ “Se P allora Q ”

$P \Leftarrow Q$ Conseguenza

$P \Leftrightarrow Q$ Doppia implicazione

1.11 Tipi di dimostrazione

- Discorsiva: si ricorre alle definizioni
- Grafica: si usa un diagramma
- Per sostituzione: si applicano leggi per sostituire elementi

Alla fine di una dimostrazione confermata si aggiunge ■.

1.12 Leggi degli insiemi

- Legge di De Morgan:

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \quad \overline{A \cap B} = \overline{A} \cup \overline{B}$$

1.13 Cardinalità

La cardinalità di un insieme finito A è il numero dei suoi elementi: $|A| = n$.

1.14 Insiemi di insiemi

$$A = \{\{2, 4, 5\}, \{3, 4, 1\}\}$$

1.15 Insieme delle parti

$$P(A) = \{x \mid x \subseteq A\}$$

Esempio: $A = \{1, 2, 3\} \implies P(A) = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}\}$

1.16 Famiglie e partizioni di insiemi

$$F = \{A_i \mid i \in I\}, \quad \bigcup F = \bigcup A_i$$

Una partizione di A è una famiglia di sottoinsiemi non vuoti, che copre A e sono a due a due disgiunti.

1.17 Prodotto cartesiano

Il prodotto cartesiano tra due insiemi A e B è:

$$A \times B = \{(a, b) \mid a \in A, b \in B\}$$

2 Relazioni tra insiemi

2.1 Definizione

Dati due insiemi A e B , una relazione R tra A e B è un sottoinsieme del prodotto cartesiano $A \times B$.

A è detto *insieme di partenza*, B *insieme di arrivo*.

Esempio:

$$A = \{a, b, c\}, \quad B = \{x, y\}$$

$$R = \{(x, a), (x, c)\} \subseteq A \times B$$

Anche la relazione vuota è possibile:

$$R = \emptyset \subseteq A \times B$$

2.2 Relazione identità

La relazione identità su un insieme A è:

$$\text{Id}_A = \{(x, x) \mid x \in A\}$$

2.3 Composizione di relazioni

Siano $R : A \leftrightarrow B$ e $S : B \leftrightarrow C$. La composizione $R; S : A \leftrightarrow C$ è definita come:

$$R; S = \{(x, z) \in A \times C \mid \exists y \in B : (x, y) \in R \wedge (y, z) \in S\}$$

Esempio:

$$A = \{x, y, z\}, \quad B = \{a, b, c, d\}, \quad C = \{1, 2\}$$

$$R = \{(x, a), (y, b), (z, c), (z, d)\}, \quad S = \{(a, 1), (a, 2), (c, 2), (d, 2)\}$$

Allora $(x, 1) \in R; S$.

2.4 Relazione opposta

Dati $R : A \leftrightarrow B$, la relazione opposta $R^{op} : B \leftrightarrow A$ è:

$$R^{op} = \{(y, x) \in B \times A \mid (x, y) \in R\}$$

$$(S^{op}; R^{op}) = (R; S)^{op}$$

2.5 Proprietà delle relazioni (TUSI)

2.5.1 Relazioni totali

$R : A \leftrightarrow B$ è totale se:

$$\forall a \in A, \exists b \in B : (a, b) \in R$$

2.5.2 Relazioni univalenti

R è univalente se:

$$\forall a \in A, \exists \text{ al massimo un } b \in B : (a, b) \in R$$

2.5.3 Relazioni suriettive (surgettive)

R è suriettiva se:

$$\forall b \in B, \exists a \in A : (a, b) \in R$$

2.5.4 Relazioni iniettive

R è iniettiva se:

$$\forall b \in B, \exists \text{ al massimo un } a \in A : (a, b) \in R$$

2.6 Risultati di dualità

$$\text{Se } R \text{ è totale} \iff R^{op} \text{ è suriettiva}$$

$$\text{Se } R \text{ è univalente} \iff R^{op} \text{ è iniettiva}$$

2.7 Teorema di caratterizzazione

$$R \text{ è totale} \iff \text{Id}_A \subseteq R; R^{op}$$

$$R \text{ è univalente} \iff R^{op}; R \subseteq \text{Id}_B$$

$$R \text{ è suriettiva} \iff \text{Id}_B \subseteq R^{op}; R$$

$$R \text{ è iniettiva} \iff R; R^{op} \subseteq \text{Id}_A$$

2.8 Composizione e proprietà

Se due relazioni R e S sono entrambe totali, allora la loro composizione $R; S$ sarà totale. Lo stesso vale per univalenza, suriettività e iniettività.

3 Funzioni

3.1 Definizione

Una funzione è una relazione che è **totale** e **univalente**.

$$f : A \rightarrow B$$

Ovvero, per ogni elemento $a \in A$ esiste esattamente un elemento $b \in B$ tale che $(a, b) \in f$.

Esempi di funzioni in informatica sono i connettivi logici.

3.2 Composizione di funzioni

Se $f : A \rightarrow B$ e $g : B \rightarrow C$, allora la composizione è:

$$f; g : A \rightarrow C$$

3.3 Funzione parziale

Una funzione parziale è una funzione che **non è totale**, quindi è solo equivalente.

3.4 Biiezione (Bigezione)

Una funzione $f : A \rightarrow B$ che è **totale**, **univalente**, **suriettiva** e **iniettiva** si chiama **biiezione**.

- Esiste una corrispondenza 1 : 1 tra gli elementi di A e B .
- Le cardinalità di A e B devono essere uguali.
- La relazione identità è una biiezione.
- La composizione di due biiezioni è una biiezione.
- L'opposta di una biiezione è una biiezione.

Se due insiemi sono in biiezione si indica:

$$A \cong B$$

3.5 Teorema di categorizzazione delle biiezioni

Sia $R : A \leftrightarrow B$. Allora:

$$R \text{ è biiezione } \iff (\text{Id}_A = R; R^{op}) \wedge (\text{Id}_B = R^{op}; R)$$

3.6 Proprietà delle biiezioni

- Simmetria:

$$A \cong B \iff B \cong A$$

- Riflessività:

$$A \cong A$$

- Transitività:

$$A \cong B \wedge B \cong C \implies A \cong C$$

4 Induzione

Gli insiemi infiniti si definiscono secondo regole di base. Tre principali usi dell'induzione:

1. Definizione induttiva di insiemi (regole per definire chi appartiene).
2. Definizione induttiva di funzioni (regole per calcolare $f(A)$).
3. Dimostrazione di proprietà.

Per definire ognuno di questi si usano delle clausole:

- **Clausola base:** stabilisce il punto di partenza.
- **Clausola induttiva (ipotesi induttiva):** definisce le regole per la propagazione.
- **Clausola terminale:** spesso implicita, indica che solo quanto definito dalle altre due clausole vale.

4.1 Definizione induttiva di un insieme

- Clausola base: certi elementi appartengono ad A .
- Clausola induttiva: come usare elementi di A per costruirne altri.
- Clausola terminale: l'insieme A non contiene altro.

Esempio: definizione dei numeri naturali

$$0 \in \mathbb{N}, \quad n \in \mathbb{N} \implies (n + 1) \in \mathbb{N}$$

Esempio: definizione di $\mathbb{N} \setminus \{5\}$

$$0 \in \mathbb{N}, \quad n \in \mathbb{N} \wedge n \neq 4 \implies (n + 1) \in \mathbb{N}, \quad n \in \mathbb{N} \wedge n = 4 \implies (n + 2) \in \mathbb{N}$$

4.2 Definizione induttiva di una funzione

- Clausola base: definire il valore dell'immagine di alcuni elementi.
- Clausola induttiva: dare una regola per calcolare l'immagine degli altri elementi.

Esempio: funzione $T : \mathbb{N} \rightarrow \mathbb{N}$

$$T(0) = 0, \quad T(n + 1) = T(n) + (n + 1)$$

$$T(4) = T(3 + 1) = T(3) + 4 = (T(2) + 3) + 4 = (T(1) + 2 + 3) + 4 = T(0) + 10 = 10$$

4.3 Definizione induttiva di una proprietà

Sia $P(m)$: " $m^2 + m$ è divisibile per 2". Vogliamo dimostrare $\forall m \in \mathbb{N}, P(m)$.

- Clausola base:

$$P(0) = 0^2 + 0 = 0 \text{ (divisibile per 2)}$$

- Clausola induttiva: se $m^2 + m$ è divisibile per 2, allora $(m+1)^2 + (m+1)$ è divisibile per 2.

$$(m+1)^2 + (m+1) = m^2 + 2m + 1 + m + 1 = m^2 + m + 2m + 2$$

Se $m^2 + m = 2k$, allora

$$(m+1)^2 + (m+1) = 2k + 2m + 2 = 2(k + m + 1)$$

Divisibile per 2, quindi la proprietà vale.

5 Relazioni su un insieme

Sia $R : A \leftrightarrow A \subseteq A \times A$.

5.1 Proprietà delle relazioni su un insieme (RATSO)

- **Riflessività:**

$$\forall a \in A : (a, a) \in R$$

L'identità Id_A e il prodotto cartesiano completo sono riflessivi. Una relazione vuota non lo è.

- **Simmetria:**

$$\forall a, b \in A : (a, b) \in R \implies (b, a) \in R$$

L'identità e il prodotto cartesiano completo sono simmetrici. La relazione vuota è simmetrica.

- **Transitività:**

$$\forall a, b, c \in A : (a, b) \in R \wedge (b, c) \in R \implies (a, c) \in R$$

L'identità e il prodotto cartesiano completo sono transitivi. La relazione vuota è transitiva.

- **Anti-simmetria:**

$$\forall a, b \in A : (a, b) \in R \wedge (b, a) \in R \implies a = b$$

L'identità è anti-simmetrica, il prodotto cartesiano completo non lo è. La relazione vuota è anti-simmetrica.

5.2 Teorema di caratterizzazione

- R è riflessiva $\iff Id_A \subseteq R$
- R è simmetrica $\iff R \subseteq R^{op}$ (oppure $R = R^{op}$)
- R è transitiva $\iff R; R \subseteq R$
- R è anti-simmetrica $\iff R \cap R^{op} \subseteq Id_A$

5.3 Chiusure di una relazione

- **Chiusura riflessiva:** $R \cup Id_A$
- **Chiusura simmetrica:** $R \cup R^{op}$
- **Chiusura transitiva:**

$$R^+ = R \cup R^2 \cup R^3 \cup \dots \cup R^n, \quad n = |A|$$

- **Chiusura transitiva unita all'identità (Stella di Kleene):**

$$R^* = R^+ \cup Id_A$$

5.4 Relazioni particolari

- **Relazione di equivalenza:** riflessiva, simmetrica e transitiva (es. Id_A , non vuota, prodotto cartesiano)
- **Relazioni di ordinamento parziale:** riflessiva, anti-simmetrica e transitiva (es. $Id_A, \leq$)

Una relazione è di ordinamento se soddisfa la proprietà di totalità:

$$\forall a, b \in A, (a, b) \in R \text{ oppure } (b, a) \in R$$

6 Grafi orientati

6.1 Definizione

Un grafo orientato è una relazione $E : V \leftrightarrow V$ su un insieme finito V :

- V : insieme dei nodi o vertici
- E : insieme degli archi

Proprietà:

- Nodi connessi da un arco sono *adiacenti*
- *Vicinato* di un nodo: nodi adiacenti
- *Vicinato in entrata*: archi che entrano
- *Vicinato in uscita*: archi che partono
- Grado in uscita d^+ , grado in entrata d^-

Hand-shaking Lemma:

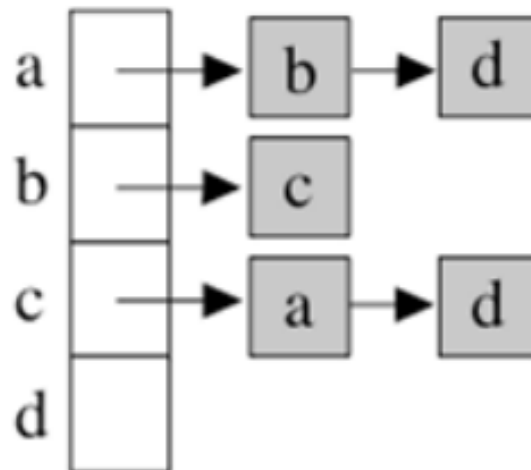
$$\sum_{x \in V} d^+(x) = |E|$$

6.2 Rappresentazione di grafi orientati

Matrice di adiacenza: $M_{ij} = M_{ji} = 1 \text{ se } i, j \in E$

	1	2	3	4	5	6
1	0	1	1	0	0	0
2	0	0	0	1	0	0
3	0	0	0	1	1	0
4	0	0	0	0	0	1
5	0	0	0	0	1	1
6	0	0	0	0	0	0

Liste di adiacenza:



6.3 Varianti di grafi

- Grafi etichettati e/o pesati: $L : (V \cup E) \rightarrow D$
- Walk: sequenza di nodi collegati da archi
- Trail: walk che non attraversa due volte lo stesso arco
- Path: walk che non attraversa due volte lo stesso nodo
- Walk chiuso: walk che inizia e finisce allo stesso nodo
- Circuito: trail che inizia e finisce nello stesso nodo
- Ciclo: path che inizia e finisce sullo stesso nodo

6.4 Connettività

- Grafo orientato fortemente connesso: per ogni $x, y \in V$ esiste un walk da x a y
- Componente fortemente connessa: sottoinsieme massimale di nodi fortemente connessi
- Grafo orientato aciclico (DAG): grafo senza cicli
- Nodi con 0 ingressi: sorgenti
- Nodi con 0 uscite: pozzi
- Arco riflessivo: cappio

6.5 Ordinamento topologico su DAG

Dato un DAG, un ordinamento topologico è una biiezione $n : V \rightarrow \{0, 1, 2, \dots, |V| - 1\}$ tale che

$$(x, y) \in E \implies n(x) < n(y)$$

Proprietà:

- Ogni DAG ha almeno una sorgente e un pozzo
- Ogni sottografo di un DAG è un DAG
- Un DAG inverso è comunque un DAG

7 Grafi non orientati

7.1 Definizione

Nei grafi orientati gli archi non sono più orientati e non sono indicati più da frecce, bensì da semplici linee connettive. Per questo, l'insieme degli archi di un grafo non orientato è un insieme di insiemi di nodi cardinalità 2.

$$P_2(V) = X \subseteq V : |X| = 2 \\ E \subseteq P_2(V)$$

Proprietà

- Un grafo non orientato non può avere cappi.
- Nodi **universali** = connessi a tutti gli altri nodi.
- Nodi **isolati** = non connessi ad alcun nodo.
- Walk, trail e path mantengono le stesse definizioni.
- Walk chiuso, circuito e ciclo mantengono le stesse definizioni.
 - Qui un walk chiuso non implica un circuito, ma un circuito indica un ciclo.
- Grafo non orientato connesso = grafo che per ogni coppia di nodi ha un walk.
- Componente connessa = insieme massimale di nodi connessi.
- Vicinato di $x = N(x) = \{y \in V : xy \in E\}$
- Grado di $x = d_x = |N(x)|$

Combinazione di circuiti: Se abbiamo due circuiti disgiunti sugli archi ma con almeno un nodo in comune possiamo unirli per formare un circuito unico.

7.2 Grafo orientato associato a un grafo non orientato

Se abbiamo un grafo non orientato $G = (V, E)$ e un grafo orientato $\overline{G} = (V, \overline{E})$ allora:

$$\overline{E} = \{(x, y) \in V \times V : x, y \in E : V \leftrightarrow V\}$$

Hand-shaking Lemma per i grafi non orientati:

$$\sum_{x \in V} d^+(x) = 2|E|$$

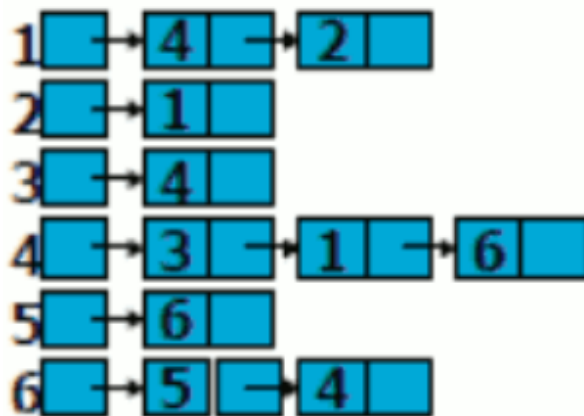
7.3 Rappresentazione di grafi non orientati

Matrice di adiacenza: $M_{ij} = M_{ji} = 1 \text{ se } i, j \in E$

La diagonale dal vertice in alto a sinistra al vertice in basso a destra sarà sempre vuota dato che non ci sono cappi.

	1	2	3	4	5	6
1	0	1	1	0	0	0
2	1	0	0	1	0	0
3	1	0	0	1	1	0
4	0	1	1	0	0	1
5	0	0	1	0	1	1
6	0	0	0	1	1	0

Liste di adiacenza: nei grafi non orientati le liste di adiacenza sono più folte, presentando simmetria (se 1 contiene 4, allora 4 contiene 1)



8 Alberi

Un **albero** è un grafo non orientato, connesso, aciclico e non vuoto. Gli alberi vengono spesso utilizzati come strutture dati per rappresentare alcune situazioni.

8.1 Caratteristiche

- Una **foresta** è un grafo non orientato, aciclico, non vuoto e non connesso.
 - Se ad un albero si toglie un arco diventa una foresta.
- I nodi alle estremità finali dell'albero (in fondo) si chiamano **foglie** ed hanno grado 1.
- Tutti i nodi con grado > 1 si chiamano nodi interni.
- Il numero dei path tra due nodi di un albero è sempre 1.
- Ogni componente connessa di un albero è un albero.

8.2 Proprietà

- Per ogni coppia di nodi distinti $x, y \in V$ esiste un unico path da x a y .
 - Il path in questione esiste perchè il grafo è connesso ed è unico perchè se non fosse unico ci sarebbe un ciclo ■.
- Per ogni arco $x, y \in E$ la rimozione di xy rende il grafo non connesso.
- Per ogni coppia di nodi distinti $x, y \in V$ tale che $xy \notin E$, l'aggiunta dell'arco xy crea un ciclo.
- Ogni albero con almeno due nodi ha almeno una foglia.
 - Dalla definizione di albero, un albero è connesso e quindi ogni nodo ha almeno un arco (grado ≥ 1) e quindi possiamo sicuramente muoverci da un nodo iniziale verso un suo vicino. Se il vicino ha grado 1 è una foglia ■, altrimenti si continua il path fino a quando non si trova la foglia prima di $|V|$ passi.■
- Ogni albero ha esattamente $n - 1$ archi con $n = |V|$
 - Togliere una foglia mantiene la struttura ad albero e riduce gli archi e i nodi entrambi di 1.
Per induzione:

$P(0) =$ non esistono alberi con 0 nodi ■

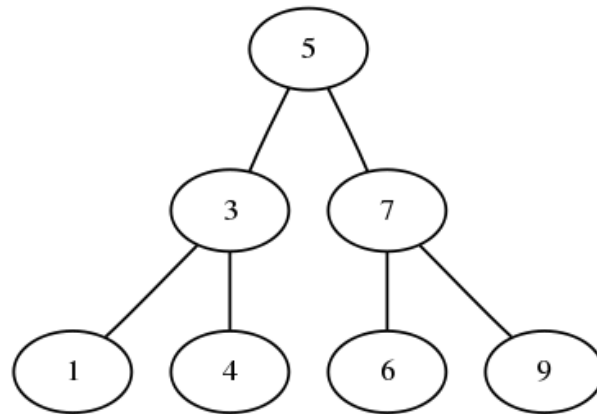
$P(1) = 0$, esiste solo un tipo di albero con 1 nodo e non può avere archi ■

$P(n) \implies P(n + 1)$ per ogni $n + 1 \geq 2$

L'ipotesi induttiva vale perchè secondo l'osservazione iniziale posso rimuovere una foglia alla volta e mantenere lo stesso rapporto tra archi e nodi. Se si continua a rimuovere foglie si ritorna eventualmente al caso base. ■

8.3 Alberi importanti

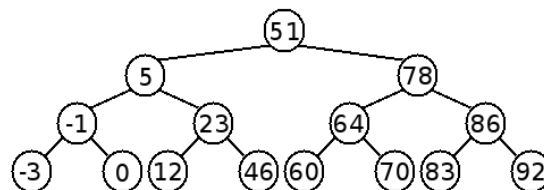
Albero radicato: Un **albero radicato** ha un nodo speciale chiamato **root** o **radice**



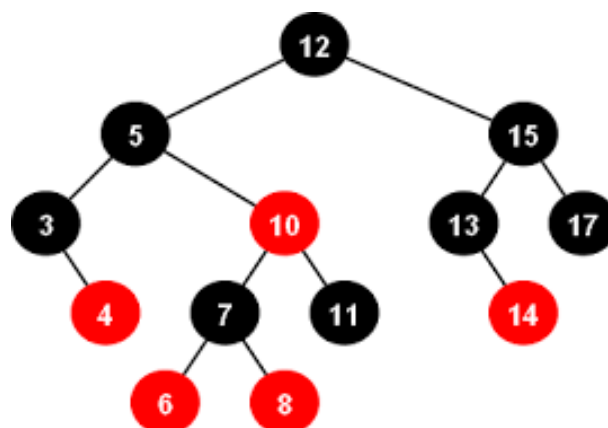
Alberi ordinali: i nodi interni hanno un ordinamento totale dei suoi figli.

Alberi cardinali: i nodi interni hanno k figli che eventualmente possono essere vuoti.

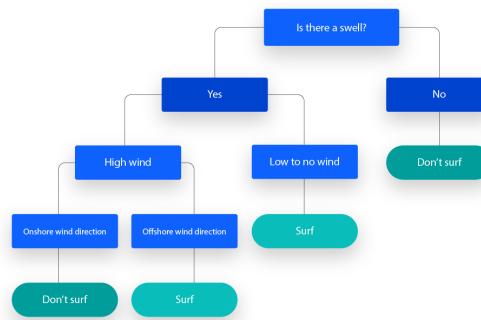
Albero binario di ricerca: grafo pesato sui nodi, ogni nodo ha al massimo due archi in uscita. Questo tipo di albero ha un ordine da sinistra a destra e permette la ricerca di valori precisi in maniera semplice. Un albero binario è un albero cardinale con $k = 2$.



Red-Black Tree:



Albero decisionale: utilizzato spesso nelle AI.



9 Cammini e cicli Euleriani e Hamiltoniani

9.1 Teorema di Eulero

Circuito euleriano: circuito che passa esattamente una volta per tutti gli archi del grafo.

Trail euleriano (o *percorso euleriano*): trail che passa esattamente una volta per tutti i nodi del grafo.

Dato un grafo non orientato e connesso:

- Esiste un trail euleriano da x a y se e solo se x e y sono gli unici nodi di grado dispari.
 - Se aggiungiamo un nuovo nodo z con archi zy e zx il grafo si trasforma e tutti i nodi avranno adesso un numero pari di archi. Prendiamo il circuito del nuovo grafo ed escludiamo i nuovi archi aggiunti, trovando il trail euleriano.
- Esiste un circuito euleriano se e solo se tutti i nodi hanno grado pari.
 - Ogni volta che si passa da un nodo ci si entra e poi ci si esce, servono quindi 2 archi ogni volta. Se si attraversa tale nodo k volte, serviranno appunto $2k$ archi. Il secondo arco del nodo da cui si parte si utilizza quando si conclude il circuito.
 - Un circuito euleriano ovviamente non può esistere se il grafo non è connesso.

Dato un grafo orientato e fortemente connesso:

- Esiste un ciclo euleriano se e solo se tutti i nodi hanno grado in ingresso uguale al grado in uscita.
- Esiste un trail euleriano da x a y se e solo se tutti i nodi hanno grado in ingresso uguale al grado in uscita e $d_x^+ = d_x^- + 1$ e $d_y^- = d_y^+ + 1$

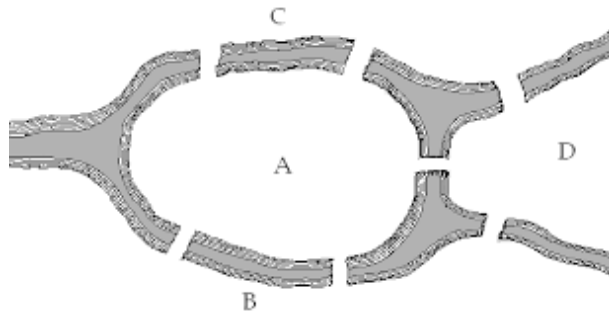
9.2 Teorema di Hamilton

Dato un grafo connesso:

- un ciclo hamiltoniano è un ciclo che passa esattamente una volta per tutti i nodi del grafo.
- un path hamiltoniano è un path che passa esattamente una volta per tutti i nodi del grafo.

Non esiste una caratterizzazione semplice come per i circuiti euleriani, ad oggi non si conosce un algoritmo efficiente ma solo brute-force.

9.3 Il problema dei sette ponti di Königsberg

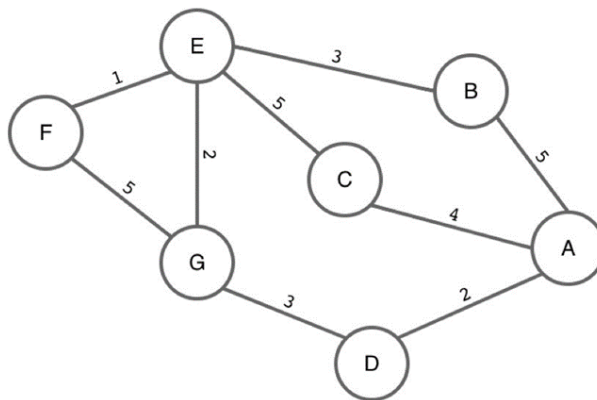


Il problema dei sette ponti di Königsberg è un famoso problema di topologia e teoria dei grafi, originariamente posto nella città di Königsberg (oggi Kaliningrad). Il problema consiste nel trovare un percorso che attraversi ciascuno dei sette ponti una sola volta e ritorni al punto di partenza.

Leonhard Euler dimostrò che tale percorso non esiste, dando così origine alla teoria dei grafi e alla definizione dei *circuiti euleriani*. La sua soluzione si basa sull'analisi dei gradi dei nodi del grafo che rappresenta la città e i suoi ponti.

9.4 Problema del commesso viaggiatore

Dato un grafo pesato non orientato, trovare un ciclo hamiltoniano di **peso minimo**, dove il peso è la somma degli archi attraversati.



10 Altro sui grafi

Diametro di un grafo: massima distanza tra coppie di nodi.

Profondità di un nodo: distanza di un nodo dalla radice.

Altezza di un nodo: massima distanza tra x e le foglie sue discendenti

Altezza di un grafo: altezza della radice

Uguaglianza tra grafi: due grafi sono uguali quando hanno gli stessi archi e gli stessi nodi.

Isomorfismo tra grafi: due grafi hanno la 'stessa forma', a seconda della loro rappresentazione. Espresso formalmente, esiste una biiezione che mappa i nodi del primo grafo confrontato con quelli del secondo.

Grafo bipartito: grafo dove ogni nodo corrisponde a un colore (non necessariamente unico) che si distingue dal colore dei suoi vicini. Questo ovviamente si verifica quando un grafo non ha cicli dispari. Gli alberi sono grafi bipartiti, in quanto basta cambiare colore ad ogni livello.

Spanning Tree: dato un grafo connesso, uno spanning tree è un albero che connette i nodi del grafo connesso con $n - 1$ archi.

10.1 Distanze

Una **distanza** su un insieme A è una funzione $d : A \times A \rightarrow \mathbb{R}$ che soddisfa le seguenti proprietà $\forall x, y, z \in A$

$$d(x, y) \geq 0$$

$$d(x, y) = 0 \iff x = y$$

$$d(x, y) = d(y, x)$$

$$d(x, y) \leq d(x, z) + d(z, y)$$

Su un grafo non orientato, la distanza tra due nodi è il numero dei passi che il walk minimo effettua da x a y .

Su un grafo orientato la condizione della simmetria non viene soddisfatta e la regola del walk minimo non vale.

11 Calcolo combinatorio

Il **calcolo combinatorio** è uno strumento molto utile nell'informatica che permette di contare e analizzare quantità.

Un utile lemma: se $P = \{A_i\}_{i \in I}$ è partizione di A allora:

$$|A| = \sum_{i \in I} |A_i|$$

Se: $\sum |A_i| > |A|$ allora almeno un $x \in A$ è stato contato due volte o più.

Se: $\sum |A_i| < |A|$ allora almeno un $x \in A$ non è stato contato.

11.1 Cardinalità di operazioni su insiemi

$$|A^n| = |A|^n$$

$$|A| = |A \setminus B| + |A \cap B|$$

$$|A \cup B| = |A \setminus B| + |A \cap B| + |B \setminus A|$$

$$|A \setminus B| = |A| - |A \cap B|$$

$$|A \cup B| = |A| + |B| - |A \cap B|$$

$$|A \cup B| \leq |A| + |B|, \text{ e sono uguali } \iff A \text{ e } B \text{ sono disgiunti}$$

Prodotto cartesiano di n insiemi

$$|A_1 \times A_2 \times \dots \times A_n| = |A_1| \cdot |A_2| \cdot \dots \cdot |A_n|$$

Unione di tre insiemi

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |A \cap C| + |A \cap B \cap C|$$

Principio di Inclusione-Esclusione

$$\left| \bigcup_{j=1}^r S_j \right| = \sum_{\substack{I \subseteq \{1,2,\dots,r\} \\ I \neq \emptyset}} (-1)^{|I|+1} \left| \bigcap_{i \in I} S_i \right|$$

11.2 Cardinalità di relazioni

Data una $R : A \leftrightarrow B$, $R \subseteq A \times B$, quanto vale $|R|$?

- $0 \leq |R| \leq |A| \cdot |B|$, sempre.
- R è totale: $|R| \geq |A|$
- R è univalente: $|R| \leq |A|$
- R è surgettiva: $|R| \geq |B|$

- R è iniettiva: $|R| \leq |B|$
- R è una funzione: $|R| = |A|$
- R è una biiezione: $|R| = |A| = |B|$
 - $|A| = |B| \iff A$ e B sono in biiezione.
 - $Fun(A, B) \cong B^{|A|}$, Fun contiene tutte le possibili funzioni $f : A \rightarrow B$, mentre $B^{|A|}$ sono le sequenze lunghe $|A|$ fatte da elementi di B .

Inoltre:

$$|Rel(A, B)| = 2^{|A| \cdot |B|}$$

Questo perchè ogni elemento di A può essere collegato con $0 \leq n \leq |B|$ elementi di B . L'insieme dei modi in cui ogni elemento di A si può collegare con B ha la stessa cardinalità di $P(B)$. Per ogni elemento significa fare questo calcolo $|A|$ volte.

11.3 Cardinalità dell'insieme delle parti

$$\forall A, \text{vale } |P(A)| = 2^{|A|}.$$

Infatti:

$$P(A) \cong Fun(A, Bool)$$

Inoltre:

$$|Fun(A, Bool)| = |Bool|^{|A|} = 2^{|A|}$$

11.4 Le targhe delle macchine italiane

Formato: XXCCCXX, con X lettera in A (alfabeto inglese escluso I, O, Q, U), e $C \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, cifra decimale.

Esistono $(22 \cdot 22 \cdot 10 \cdot 10 \cdot 10 \cdot 22 \cdot 22)$ targhe diverse, ovvero 234 milioni circa.

Questo è un esempio classico dell'applicazione del calcolo combinatorio, in particolare l'equivalenza riguardante il prodotto cartesiano riportato sopra.

11.5 Permutazioni, disposizioni e combinazioni

Dato un insieme di n oggetti, determinare in quanti modi questi oggetti si possono raggruppare.

11.5.1 Permutazione

Sia A un insieme con $|A| = n$, una permutazione di A è una sequenza ordinata di tutti gli elementi di A .

Una permutazione di A è una biiezione:

$$\pi : A \rightarrow |A|$$

$Perm(A)$ = insieme delle permutazioni possibili di A .

$$|Perm(A)| = |A|!$$

$$|A| = |B| \implies |Perm(A)| = |Perm(B)|$$

$$|Bii(A, B)| = \dots$$

$$0 \text{ se } |A| \neq |B|$$

$$|A| \text{ se } |A| = |B|$$

$$|A| = |B|$$

Anagrammi possibili scambi di lettere di lettere in una parola.

Gli anagrammi possibili di una parola con tutte lettere diverse sono:

$$|Anagrammi| = |lettereParola|!$$

Generalmente però non tutte le parole hanno lettere diverse, per questo si stabilisce questa come regola generale:

$$|Anagrammi| = \frac{|lettereParola|!}{c_1! \cdot c_2! \cdot \dots \cdot c_n!}$$

Con:

c_i = numero di occorrenze di una lettera nella parola

11.5.2 Disposizioni

Sia A un insieme con $|A| = n$, una **disposizione** degli elementi A in k posti è una sequenza ordinata di k elementi distinti in A .

$$a_1, a_2 \dots a_k$$

Per trovare il numero totale di disposizioni in k posti si fa così:

$$D(n, k) = \frac{n!}{(n - k)!}$$

oppure:

$$D(n, k) = \frac{Perm(A)}{Perm(B)} \text{ con } |B| = n - k$$

Visivamente, si fanno tutte le permutazioni e prendo solo i primi k elementi di ogni permutazione.

11.5.3 Combinazioni

Sia A un insieme con $|A| = n$, una **combinazione** di k elementi di A è un sottoinsieme di A di cardinalità k .

L'insieme delle combinazioni di k elementi di A è $P_k(A)$

In pratica, le combinazioni sono come le disposizioni, ma non tengono conto dell'ordine, ogni combinazione è un insieme diverso.

Per trovare il numero totale di combinazioni di un insieme si fa così:

$$C(n, k) = \frac{D(n, k)}{Perm(k)}$$

Oppure:

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!}$$

Il numero totale delle combinazioni di un insieme si chiama **coefficiente binomiale**.

11.5.4 Cardinalità sfruttando regole di biiezione

Disporre z magliette diverse in g cassetti.

Rappresentiamo le magliette con 0 e il separatore dei cassetti con 1 formando questa stringa d'esempio:

0010111000

Il totale dei modi in cui si possono disporre quindi in n modi diversi, con $n =$ totale stringhe realizzabili.

In ogni stringa di queste ci sono z zeri e $g - 1$ separatori.

$$n = \binom{z + g - 1}{g - 1}$$

Nel caso le magliette fossero numerate, ovvero distinte, la formula è questa:

$$n = g^z$$

11.6 Calcolo combinatorio sui grafi

Il numero massimo grafi non orientati con n nodi è:

$$\sum_{i=0}^m \binom{m}{i} = \binom{m}{0} + \binom{m}{1} + \dots + \binom{m}{i} = 2^m$$

Con:

$$m = \binom{n}{2} = \frac{n(n-1)}{2} = \text{numero massimo di archi su } n \text{ nodi}$$

$i =$ numero di archi

Esistono 2^m grafi non orientati con n nodi perché ogni arco ha due valori, o esiste o non esiste.

Nei grafi orientati invece abbiamo n^2 come numero massimo di archi.

Quanti grafi non isomorfi con n nodi esistono?

Purtroppo non si conosce la formula, va calcolato "a mano".

Pagina web esplicativa del problema.

Complemento di un grafo: $G(V, E)$ è il complemento di $H(V, E')$ se:

$$E' = (V \times V) \setminus E = \{xy \in (V \times V) : xy \notin E\}$$

Ovvero se H contiene gli archi che mancano a G .

In generale, il complemento è una relazione:

$$C \subseteq G_n \times G_n$$

C è una relazione:

- totale
- surgettiva
- simmetrica
- biiettiva

Shortest path tra due nodi: path di lunghezza minore tra due nodi.

Gli shortest path NON sono unici.

In una cricca K con n nodi:

- Tra x e y esiste un solo shortest path, ovvero xy
- Esistono $|Perm(K)|$ path hamiltoniani, ovvero $n!$
- Esistono $n!$ cicli hamiltoniani.
 - Esistono $\frac{n!}{2n}$ cicli hamiltoniani non equivalenti (che non usano lo stesso insieme di archi), bisogna infatti escludere le rotazioni e le simmetrie.
- Presi k nodi di questa cricca, esistono $\frac{n!}{(n-k)!}$ path diversi tra due nodi.
- Esistono $n! < x < \frac{n(n-1)}{n-1}$ spanning tree, infatti sono sempre maggiori del numero dei path hamiltoniani, in quanto questi ultimi sono particolari tipi di spanning tree. La formula più gettonata è n^{n-2} .

In un albero di altezza h :

- La distanza massima tra due nodi x, y è $2h$.
- Se l'albero è un binario completo (ogni nodo ha 2 figli tranne quelli al livello h), il numero delle foglie totali è 2^h e il numero di nodi è uguale a $\sum_{k=0}^h 2^k$, ovvero $2^{h+1} - 1$ (il doppio delle foglie).
- Se l'albero è binario con f foglie, la minima altezza di h è $\log_2 f$ (arrotondato all'intero superiore) e ci sono $f - 1$ biforcazioni (nodi con più di un figlio).

12 Induzione strutturale

12.1 Liste

Strutture dati che consentono di memorizzare **sequenze omogenee** di dati di **lunghezza variabile**.

Una lista viene indicata così:

$$[a_1, a_2, \dots, a_k]$$

La lista vuota è:

$$[]$$

Per indicare che un elemento fa parte di una lista si può usare anche questa struttura:

$$a : lst$$

Nelle liste conta l'ordine e può esserci molteplicità, al contrario degli insiemi.

Lunghezza di una lista:

$$CB = len([]) = 0$$

$$CI = len(a : lst) = len(lst) + 1$$

Somma di una lista:

$$CB = sumList([]) = 0$$

$$CI = sumList(a : lst) = sumList(lst) + a$$

Appartenenza a una lista:

$$CB = belList([], b) = false \forall b \in A$$

$$CI = sumList(a : lst, b) = true, \forall b \in A : a = b$$

$$CI = sumList(a : lst, b) = belList(lst, b), \forall a, b \in A : a \neq b$$

Ora iniziamo a osservare le induzioni strutturali.

Concatenazione tra liste:

$$CB = append([], L_2) = L_2$$

$$CI = append(a : L_1, L_2) = a : append(L_1, L_2)$$

Principio di induzione delle liste:

$$CB = [] \in L_A$$

$$CI = (a \in A \wedge lst \in L_A) \implies a : lst \in L_A$$

12.2 Induzione strutturale su alberi binari

Definizione induttiva degli alberi binari:

$$CB = \lambda \text{ (albero vuoto)} \in BT$$

$$CI = t_1 \in BT \wedge t_2 \in BT \implies N(t_1, t_2) \in BT$$

Dimensione:

$$CB = size(\lambda) = 0$$

$$CI = size(N(t_1, t_2)) = size(t_1) + size(t_2) + 1$$

Altezza:

$$CB = height(\lambda) = -1$$

$$CI = height(N(t_1, t_2)) = \max(height(t_1), height(t_2)) + 1$$

Appartenenza:

$$CB = belBT(\lambda, a) = false$$

$$CI = belBT(N(t_1, b, t_2), a) = true(a = b), belBT(t_1, b) \vee belBT(b, t_2)(a \neq b)$$

Visita:

$$CB = visit(\lambda) = []$$

$$CI = visit(N(t_1, a, t_2)) = a : app(visit(t_1), visit(t_2))$$

12.3 F-Terms

Astrazione di tutte le regole di induzione strutturale, induzione strutturale generalizzata. Una **segnatura** è una famiglia di simboli indicizzata da $\mathbb{N}$. $\forall n \in \mathbb{N}, f = \{f_n\}_n \in N$, f_n = insieme di simboli con **arietà** n .

Simboli con arietà = 0 sono costanti.

Data f come segnatura, l'insieme $f\text{-termini}(fTerm)$ è il più piccolo insieme che soddisfa:

Per ogni costante: $c \in f_0, c \in fTerm$

$\forall n \geq 1, \forall f \in f_n$, se $t_1 \cdots t_n \in fTerm$ allora: $f(t_1 \cdots t_n) \in fTerm$.

Liste come segnature:

$$L_0^A = \{[]\}$$

$$L_1^A = \{a : |a \in A\}$$

$$L_2^A = \emptyset, \forall n \geq 2$$

Alberi binari come segnature:

$$BT_0 = \{\lambda\}$$

$$BT_1 = \emptyset$$

$$BT_2 = \{N\}$$

$$BT_n = \emptyset, \forall n \geq 3$$

12.4 Principio di induzione strutturale

Sia F una segnatura e P una proprietà su $FTerm$, $P : FTerm \rightarrow Bool$.

Se (**caso base**) $P(c)$ è vera per ogni simbolo costante

e (**passo induttivo**) se per ogni $n \geq 1$, $\forall f \in F_n$ e per tutti i termini $t_1 \cdots t_n \in FTerm$ vale che se $P(t_1) \cdots P(t_n)$ sono vere allora anche $P(f(t_1 \cdots t_n))$ lo è...

Allora $P(t)$ è vera per ogni $t \in FTerm$.

13 Ricorsione

La **ricorsione** è molto legata all'induzione strutturale.

Consiste nel definire qualcosa nel termine di se stesso.

Concettualmente, ci sono dei casi di ricorsione che funzionano e altri che non funzionano.

Per esempio, l'induzione è un caso di ricorsione che funziona, è sempre ben data e ben fondata.

Definizione di una funzione ricorsiva:

$$rec(n) = \{n \in D_1 \implies espr_1, n \in D_2 \implies espr_2, \dots, n \in D_k \implies espr_k\}$$

Attenzione, se $n \in D_1 \cap D_2$ per esempio allora non si parla di funzione, ma relazione non univalente.

Intuitivamente, se una funzione ha delle definizioni che iterano all'infinito non è una funzione, in quanto tali casi non ritorneranno mai niente e non vengono inclusi verso la totalità della funzione (funzione parziale).

Una relazione / funzione è **ben fondata** se non esiste una catena infinita decrescente di elementi.

14 Linguaggi Formali

Esistono due tipi di linguaggio:

- **Linguaggi naturali:** come l'italiano, anche se si commettono errori si può sempre capire.
- **Linguaggi formali:** strettamente definito, se si commettono errori non si comprende.

La **sintassi** definisce le regole del linguaggio (frasi ammissibili) e la **semantica** definisce il significato delle varie frasi.

Un **linguaggio** è l'insieme di tutte le **stringhe** ammissibili su un certo **alfabeto**, ovvero un insieme *finito* di simboli.

Una stringa su un alfabeto è una sequenza di lunghezza arbitraria (finita) di simboli dell'alfabeto.

Nei **linguaggi di programmazione** le stringhe ammissibili sono i **programmi**.

Dato L (linguaggio) su A (alfabeto):

$$L \subseteq A^*$$

La grammatica è una **tecnica generativa** e permette di costruire le stringhe, invece un **automa** controlla la validità delle stringhe generate.

L'insieme di tutti i possibili linguaggi su A è:

$$P(A^*)$$

14.1 Automa

Dato un alfabeto A , un **automa a stati finiti** su A è una tripla:

$$\mathcal{A} = (S, T, F)$$

Dove:

S = insieme finito, l'insieme degli stati.

$T \subseteq (A^*S)XS$ è la relazione di transizione

$F \subseteq S$ è l'insieme di stati finali (o di accettazione)

I grafi importanti per l'informatica sono gli automi di **Mealy** e **Moore** e la **macchina di Turing**. Un **automa a stati finiti** è circa un **multigrafo** (accetta più archi uguali) orientato ed etichettato.

Ecco l'idea:

- Si legge una stringa
- Analizza lo stato di un carattere
- Si passa ad un nuovo stato

I nodi si chiamano **stati** e gli archi si chiamano **transizioni**.

L'automa convalida una stringa se esiste un walk corrispondente alla stringa nell'automa.

Stato di accettazione: stato in cui la stringa viene verificata, stato finale, si indica così:

$$\overline{\text{stato}}$$

Raggiungibilità: sia $\mathcal{A} = (S, T, F)$ un automa sull'alfabeto A , $\forall a \in A$:

$$T_a = (x, y) | ((x, a)y) \in T \in \text{Rel}(S, S)$$

In pratica, si indicano le transizioni necessarie per l'accettazione di una stringa.

Linguaggio accettato da uno stato: sia $\mathcal{A} = (S, T, F)$ un automa sull'alfabeto A e $x \in S$, il **linguaggio accettato da x** è:

$$\langle\langle x \rangle\rangle = w \in A^* | \exists y \in F, (x, y) \in T_w$$

In questo caso si parte dallo stato x . Le stringhe accettate devono terminare in uno stato di accettazione.

Esercizio: progettare un automa che accetti solo le stringhe che cominciano con "al", l'alfabeto è quello latino minuscolo.



Con stato "2" come stato di accettazione.

14.2 Automi deterministici e non deterministici

Automa deterministico: un automa è deterministico se e solo se la relazione di transizione è una funzione (quindi totale e univalente).

Automa non deterministico: un automa è non deterministico se la relazione di transizione non è una funzione, ovvero c'è almeno uno stato in cui partono più di una transizione che ritornano lo stesso simbolo (non univalente) oppure c'è almeno uno stato a cui non sono collegate transizioni per tutti i simboli (non totale).

14.3 Grammatiche

Una **grammatica** è un insieme di regole che definisce come comporre le frasi di un linguaggio.

Una grammatica è una coppia (S, P) , con S come insieme dei simboli non terminali e P come insieme delle **produzioni**. In una grammatica sono presenti simboli:

- **Terminale:** simbolo considerato finale, che di per sè non allunga la stringa.
- **Non terminale:** simbolo che oltre ad un simbolo aggiunge altri simboli a scelta in posizioni ben precise.

Ogni regola di una grammatica si chiama **produzione** ed hanno la forma:

$$Testa \rightsquigarrow Corpo$$

La testa è un simbolo terminale, il corpo una sequenza di simboli terminali e non. La freccia utilizzata nella definizione sopra è errata, quella giusta non è presente in Latex (quella giusta si chiama "potrebbe essere").

Albero di derivazione sintattica (Parse Tree): albero che mostra la composizione di una stringa.

Non sempre esiste un automa a stati finiti in grado di definire una grammatica. Per esempio, questa grammatica non possiede un automa rappresentativo:

$$\{a^n b^n | n \in N\} \text{ su alfabeto } A = \{a, b\}$$

15 Logica

La **logica** è la disciplina che studia la correttezza.

Nella seconda metà del 1800 sono state sviluppate notazioni matematiche per gli operatori logici.

Tutto il calcolo dei computer si basa sulla logica.

La logica ci serve per esprimere concetti in modo non ambiguo.

Proposizione: frase in linguaggio naturale che soffisca:

- **Prinipio del terzo escluso:** o è vera o è falsa
- **Principio di non contraddittorietà:** non entrambi (vera e falsa)

Ogni proposizione è definita da una formula.

Quando una formula segue un modello si rappresenta così:

$$\models P$$

Tautologia: formula proposizionale che è vera in ogni intepretazione, sono di modello:

$$\models P$$

Contraddizione: formula proposizionale che è falsa in ogni interpretazione, sono di modello:

$$\models \neg P$$

Conseguenza logica: più formule (insieme Γ) convalidano un'altra formula, ovvero se tutte insieme seguono un modello:

$$A \wedge B, B \wedge C \models A \wedge C$$

La dimostrazione valida per la conseguenza logica (Q è dimostrabile da Γ in R , ovvero l'insieme delle regole di inferenza) spiega che ogni formula in Γ :

- O è un assioma / ipotesi in Γ e quindi è vera.
- O è ottenuta tramite regole di inferenza, e quindi è vera.

Principio di sostituzione: possono esistere regole "r" che dicono che se sono vere le formule $P_1, P_2, \dots, P_n$ allora è vera P :

$$\frac{P_1 \dots P_n}{P} [r]$$

Esistono delle formule vere **a prescindere**, sempre, si chiamano **assiomi**.

$$\overline{P_1} [r]$$

Ogni **proof system** può essere **completo**, ma è sempre **corretto**:

- **Corretto:** $\Gamma \vdash_R P \implies \Gamma \models P$
- **Completo:** $\Gamma \models P \implies \Gamma \vdash_R P$

Quando due formule / formule proposizionali sono **logicamente identiche** (hanno lo stesso output per ogni input) si indica così:

$$P1 \equiv P2$$

15.1 Tecniche di dimostrazione

Contronominale: dimostrazione dell'inverso della tesi (tutto col not)

Dimostrazione per assurdo: dimostrazione che la tesi non può non essere vera o che il contrario della tesi è sempre falso

15.2 Logica dei predicati

"Se Anna è la mamma di Bruno e Bruno ha figli, allora Anna è nonna"

Questa è una proposizione, ma nel linguaggio matematico / logico è difficile realizzarlo, possiamo fare così:

$$\text{mamma}(\text{Anna}, \text{Bruno}) \wedge \text{haFigli}(\text{Bruno}) \implies \text{nonna}(\text{Anna})$$

Queste "funzioni" che ritornano Intuitivamente solo valori booleani si chiamano **predicati**.

Quantificatori: simboli che indicano pronomi o aggettivi come "tutti", "nessuno", "alcuni"... Indicano le quantità.

$\forall$ = per ogni (tutti)

$\exists$ = esiste (almeno 1, alcuni)

$\nexists$ = non esiste (nessuno)

L'uso dei quantificatori Q assume questa forma:

$$Qx.P \text{ oppure } Qx : P$$

Con P come formula proposizionale.

Alfabeti del primo ordine: un alfabeto del primo ordine A è una quadrupla (C, F, P, V) dove:

- C = costanti (Anna, Bruno...)
- F = funzioni (padre, altezza... (input: dominio, output: dominio))
- P = predicati (input: dominio, output: bool)
- V = variabili (x,y,pippo...)